

Agentic AI Chapter 6: Memory, Guardrails, LLM Gateways & Evaluations

Core Learning Objective: Productionize AI agents with enterprise-grade resilience. Implement short- and long-term memory architectures, build input/output guardrails against prompt injection, configure multi-provider LLM gateways, and establish automated evaluation frameworks (RAG Triad).

1. The Multi-Tier Memory Hierarchy

An agent without memory treats every user interaction as its first day on earth. Production agents implement a **two-tier memory architecture**:

```
flowchart TD
    Memory["Agent Memory Architecture"]

    subgraph Short_Term ["1. Short-Term Working Memory (Session Scope)"]
        Buffer["Sliding Window Buffer (Last K messages)"]
        Summarizer["Summary Memory (LLM condenses older turns)"]
    end

    subgraph Long_Term ["2. Long-Term Persistent Memory (Cross-Session)"]
        Semantic["Semantic Memory (User facts, preferences, knowledge base)"]
        Episodic["Episodic Memory (Past successful tool runs & past failures)"]
    end

    Memory --> Short_Term
    Memory --> Long_Term
```

Implementing Fact-Extraction Long-Term Memory in Python:

```
import json

class LongTermUserMemory:
    def __init__(self):
        self.profile = {} # In prod: PostgreSQL JSONB or Redis

    def extract_and_store_facts(self, user_message: str):
        # Simulated extraction of persistent user attributes
        if "i live in" in user_message.lower():
            city = user_message.split("in")[-1].strip().title()
            self.profile["location"] = city
        if "allergic to" in user_message.lower():
            allergy = user_message.split("to")[-1].strip()
            self.profile["allergy"] = allergy

    def get_context_injection(self) → str:
        if not self.profile:
            return ""
        return f"\n[User Profile Context]: {json.dumps(self.profile)}\n"

mem = LongTermUserMemory()
mem.extract_and_store_facts("Hi, I live in Seattle and I am allergic to peanuts.")
print("Injected Memory Context:")
print(mem.get_context_injection())
```

2. Guardrails: Defending Against Prompt Injections & PII Leaks

```
flowchart LR
    UserInput["User Input"] --> InputGuard["Input Guardrail<br/>- PII Masking (SSN, Cards)"]
    InputGuard -->|Safe| AgentCore["Agent Reasoning Core (LLM)"]
    InputGuard -->|Malicious / Jailbreak| Block["Immediate Rejection (400 Bad Request)"]
    AgentCore --> OutputGuard["Output Guardrail<br/>- Hallucination Verification<br/>- To"]
    OutputGuard --> SafeResponse["Sanitized Response to User"]
```

Prompt Injection & PII Redactor:

```
import re

class InputGuardrail:
    PII_REGEX = {
        "email": r"[\w\.-]+@[\w\.-]+\.\w+",
        "credit_card": r"\b(?:\d{4}[- ]?)\d{3}\d{4}\b",
        "ssn": r"\b\d{3}-\d{2}-\d{4}\b"
    }

    INJECTION_SIGNALS = [
        "ignore previous instructions",
        "system prompt override",
        "you are now DAN",
        "disregard all rules"
    ]

    def sanitize_and_validate(self, text: str) → str:
        # 1. Injection detection
        lower = text.lower()
        for signal in self.INJECTION_SIGNALS:
            if signal in lower:
                raise ValueError(f"Security Alert: Malicious prompt injection detected ('{signal}')")

        # 2. PII Redaction
        sanitized = text
        for pii_type, pattern in self.PII_REGEX.items():
            sanitized = re.sub(pattern, f"[REDACTED_{pii_type.upper()}]", sanitized)

        return sanitized

guard = InputGuardrail()
safe_input = guard.sanitize_and_validate("Contact me at user@corp.com with card 4111-2222-3333-4444")
print("Sanitized Prompt:", safe_input)
```

3. Production LLM Gateways (LiteLLM Architecture)

Never call proprietary model APIs directly in production microservices. Route all traffic through a **Unified LLM Gateway**:

flowchart TD

```
App["Internal Microservices"] → Gateway["LLM Gateway (LiteLLM / Envoy)"]
```

```
Gateway → CheckBudget{"Budget & Rate Limit OK?"}
```

```
CheckBudget → |No| Reject["429 Rate Limit"]
```

```
CheckBudget → |Yes| Route["Routing & Failover"]
```

```
Route → Primary["Primary Provider (OpenAI GPT-4o)"]
```

```
Primary --> |Timeout / 503 Outage| Fallback["Automatic Fallback (Anthropic Claude 3.5)"]
```

Key Gateway Responsibilities:

1. **Multi-Provider Fallback:** If OpenAI returns a 500 or times out, failover instantly to Claude or Gemini with zero user downtime.
2. **Cost & Token Tracking:** Monitor expenditure per department/team in real time.
3. **Semantic Caching:** Cache identical prompts in Redis, reducing API costs by 30-50%.

4. The RAG Evaluation Triad

How do you scientifically benchmark whether your RAG agent is improving? Use the **RAG Triad** (Ragas / TruLens):

flowchart TD

```
UserQuery["1. User Query"]
```

```
Context["2. Retrieved Context Chunks"]
```

```
Answer["3. LLM Generated Answer"]
```

```
UserQuery ↔ |Context Relevance: Are chunks relevant to question?| Context
```

```
Context ↔ |Groundedness / Faithfulness: Is answer factual to context?| Answer
```

```
UserQuery ↔ |Answer Relevance: Does answer address the query?| Answer
```

1. **Context Relevance:** Evaluates if the retrieved chunks contain the actual facts needed to answer the question without irrelevant noise.

2. **Faithfulness (Groundedness):** Evaluates whether every claim in the generated answer can be directly inferred from the retrieved context (Score \$1.0\$ = zero hallucinations).
3. **Answer Relevance:** Evaluates whether the answer directly and concisely satisfies the user's intent.